| Course Name: | Competitive Programming Lab | Semester: | IV |
|---|---|---|---|
| Date of Performance: | 18/ 03/ 2026 | DIV/ Batch No: | B2 |
| Student Name: | Ashwera Hasan | Roll No: | 16010124107 |

# Experiment No: 5

## Title: Bit Manipulation Techniques

**Aim of the Experiment:** To understand and Bit Manipulation Techniques for a given problem

**Objective of the Experiment:**
To understand and apply bit manipulation techniques for efficient data processing by performing operations such as setting, clearing, toggling, and checking specific bits using bitwise operators.

**COs to be achieved:**

**CO 2 Analyse and optimize algorithms using amortized analysis and bit manipulation, equipping them to tackle complex computational problems.**

**Theory:**

Bit manipulation refers to the act of directly operating on individual bits of binary data. Since computers internally represent data in binary (0s and 1s), bit manipulation allows programs to run faster, use less memory, and perform low-level operations efficiently.

Bitwise operators work at the bit level rather than on entire numbers. These operations are commonly used in embedded systems, cryptography, compression algorithms, networking, and performance-critical applications.

Common Bitwise Operators:

| Operator | Name | Description |
|---|---|---|
| & | AND | Sets bit to 1 if both bits are 1 |
| \| | OR | Sets the bit to 1 if **at least one** of the corresponding bits is 1 |
| ^ | XOR | Sets bit to 1 if bits are different |
| ~ | NOT | Inverts all bits |
| << | Left Shift | Shifts bits left (multiplies by 2) |
| >> | Right Shift | Shifts bits right (divides by 2) |

**Advantages of Bit Manipulation**

- Faster execution compared to arithmetic operations
- Efficient memory usage
- Useful for low-level hardware interaction
- Simplifies flag handling and permissions

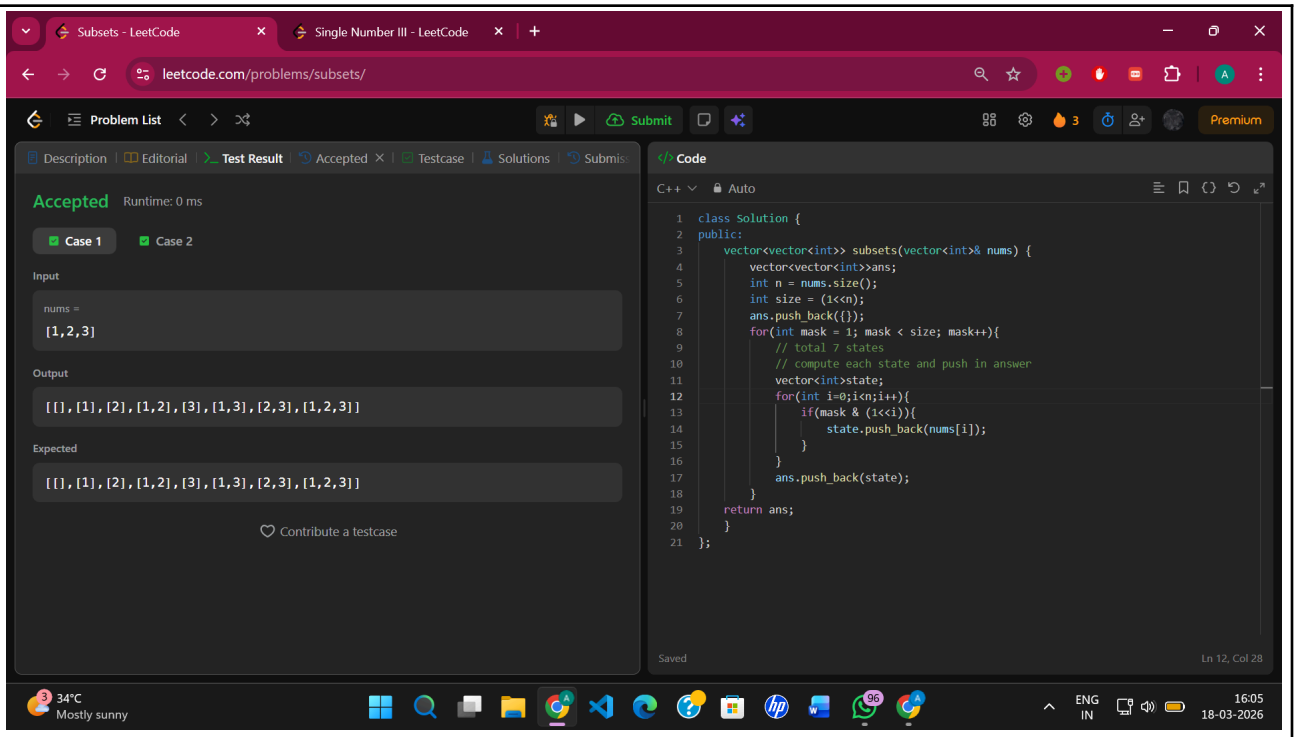
**Applications**

- Embedded systems and microcontrollers
- Cryptography and security algorithms
- Network protocol implementation
- Data compression
- Game development and graphics

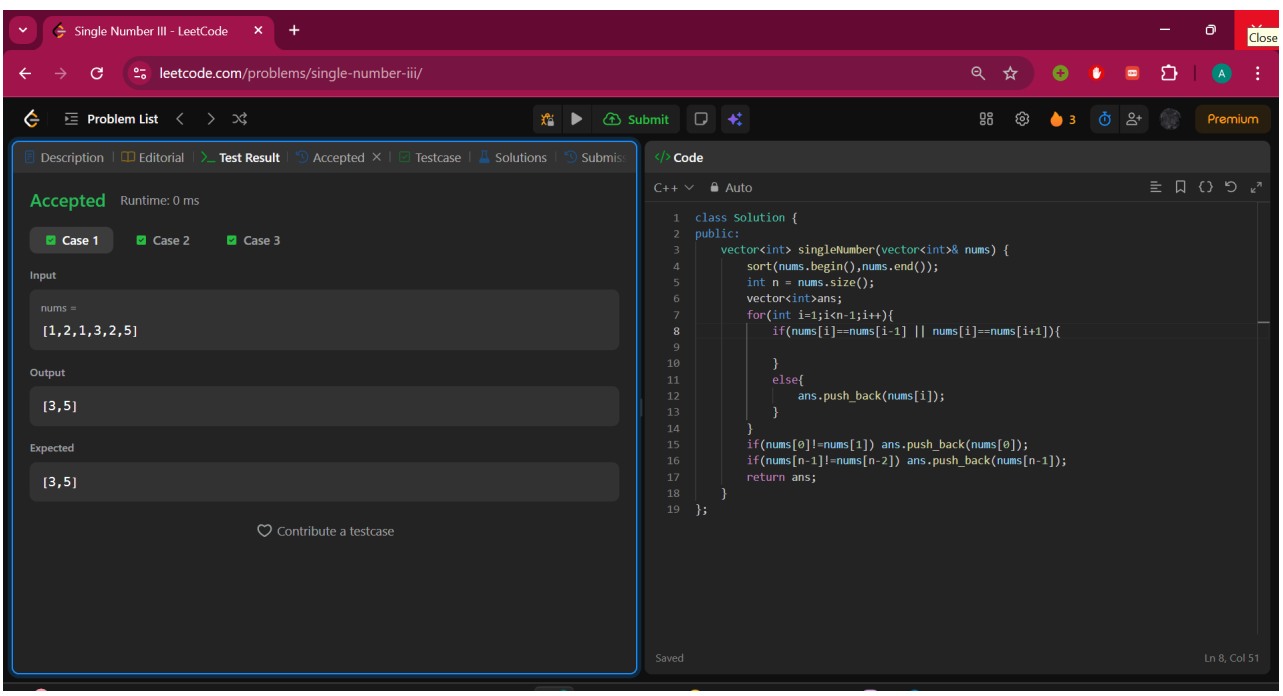
**Problem Statements:**

| Difficulty (as per lab) | Problem | LeetCode mapping |
|---|---|---|
| Easy | Two unique elements (others twice) | LC 260 – Single Number III |
| Medium | Generate all subsets using bitmasks | LC 78 – Subsets |
| Difficult | Bitmask DP for assignment ($N \leq 12$) | LC 1879 – Minimum XOR Sum of Two Arrays |

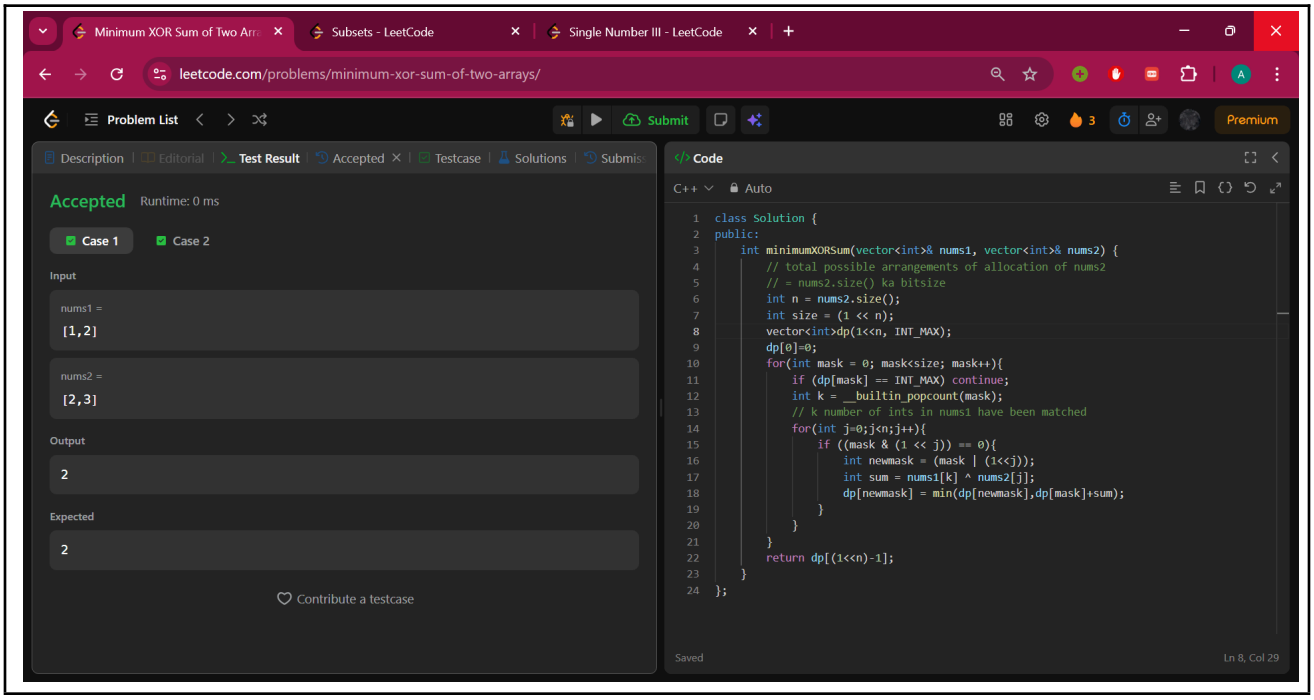
| **Code :** |
|---|
| Easy: |


Subsets - LeetCode
Single Number III - LeetCode
leetcode.com/problems/subsets/
Problem List
Submit
Premium
Description
Editorial
Test Result
Accepted
Testcase
Solutions
Code
C++
Auto
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[1,2,3]
Output
[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
Expected
[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
Contribute a testcase
class Solution {
public:
vector<vector<int>> subsets(vector<int>& nums) {
vector<vector<int>>ans;
int n = nums.size();
int size = (1<<n);
ans.push_back({});
for(int mask = 1; mask < size; mask++){
// total 7 states
// compute each state and push in answer
vector<int>state;
for(int i=0;i<n;i++){
if(mask & (1<<i)){
state.push_back(nums[i]);
}
}
ans.push_back(state);
}
return ans;
}
};
Saved
Ln 12, Col 28
34°C
Mostly sunny
ENG
IN
16:05
18-03-2026

Medium:

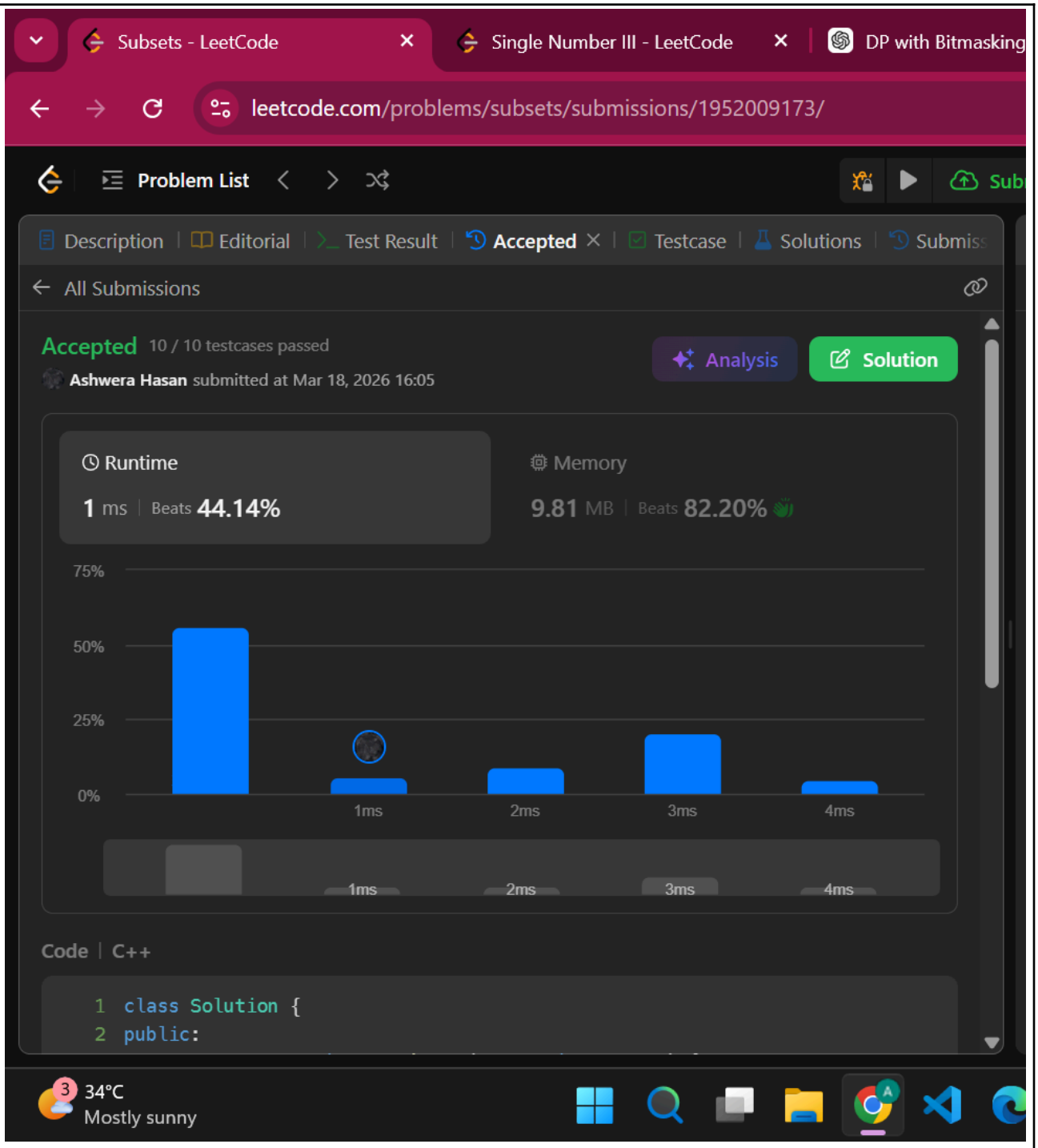

Single Number III - LeetCode
leetcode.com/problems/single-number-iii/
Problem List
Submit
Premium
Description
Editorial
Test Result
Accepted
Testcase
Solutions
Code
C++
Auto
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,2,1,3,2,5]
Output
[3,5]
Expected
[3,5]
Contribute a testcase
class Solution {
public:
vector<int> singleNumber(vector<int>& nums) {
sort(nums.begin(),nums.end());
int n = nums.size();
vector<int>ans;
for(int i=1;i<n-1;i++){
if(nums[i]==nums[i-1] || nums[i]==nums[i+1]){
}
else{
ans.push_back(nums[i]);
}
}
if(nums[0]!=nums[1]) ans.push_back(nums[0]);
if(nums[n-1]!=nums[n-2]) ans.push_back(nums[n-1]);
return ans;
}
};
Saved
Ln 8, Col 51

Hard:

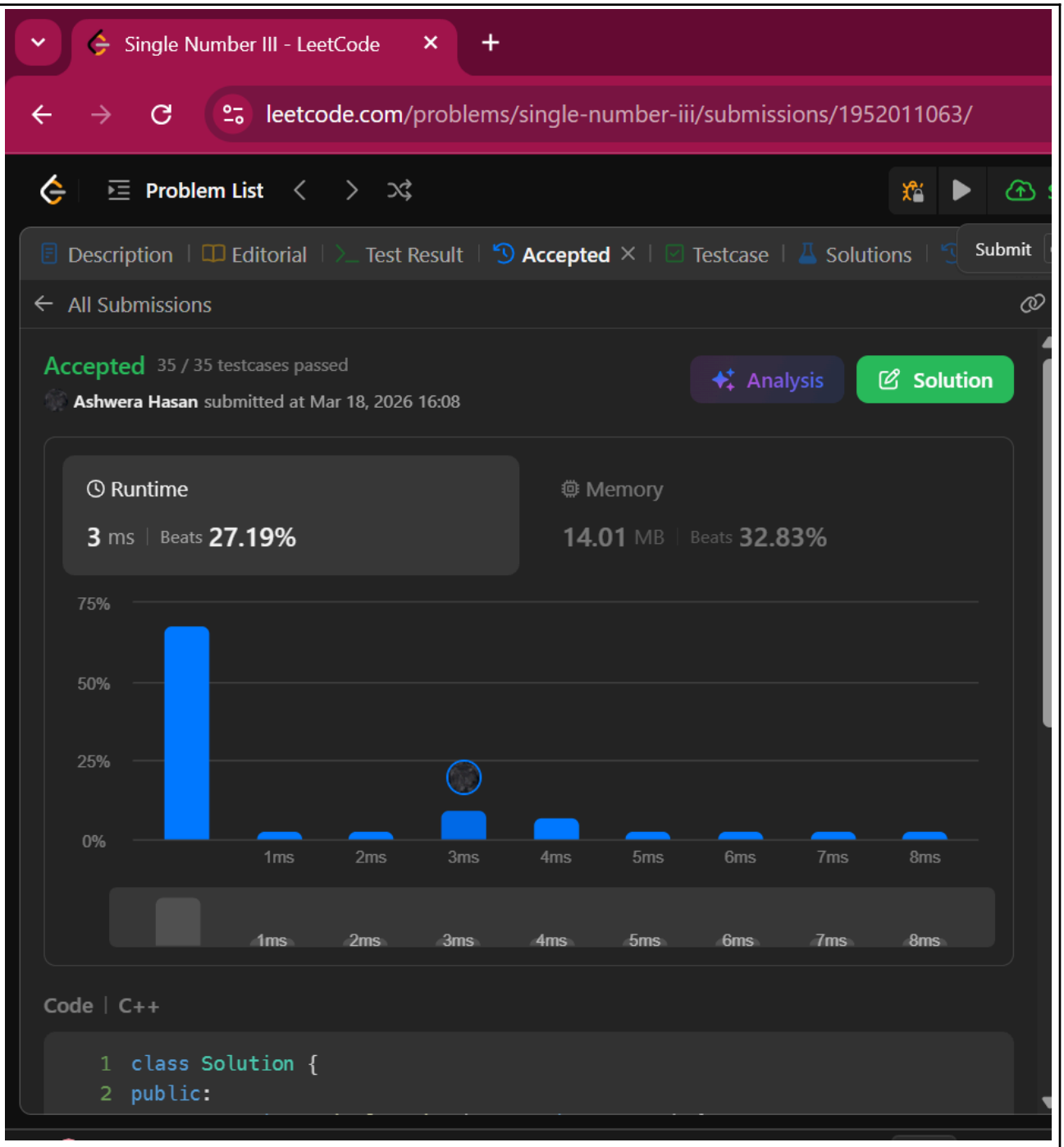

Minimum XOR Sum of Two Arr
Subsets - LeetCode
Single Number III - LeetCode
leetcode.com/problems/minimum-xor-sum-of-two-arrays/
Problem List
Submit
Premium
Description
Editorial
Test Result
Accepted
Testcase
Solutions
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums1 =
[1,2]
nums2 =
[2,3]
Output
2
Expected
2
Contribute a testcase
Code
C++
Auto
class Solution {
public:
int minimumXORSum(vector<int>& nums1, vector<int>& nums2) {
// total possible arrangements of allocation of nums2
// = nums2.size() ka bitsize
int n = nums2.size();
int size = (1 << n);
vector<int>dp(1<<n, INT_MAX);
dp[0]=0;
for(int mask = 0; mask<size; mask++){
if (dp[mask] == INT_MAX) continue;
int k = __builtin_popcount(mask);
// k number of ints in nums1 have been matched
for(int j=0;j<n;j++){
if ((mask & (1 << j)) == 0){
int newmask = (mask | (1<<j));
int sum = nums1[k] ^ nums2[j];
dp[newmask] = min(dp[newmask],dp[mask]+sum);
}
}
}
return dp[(1<<n)-1];
}
};
Saved
Ln 8, Col 29

## Output:

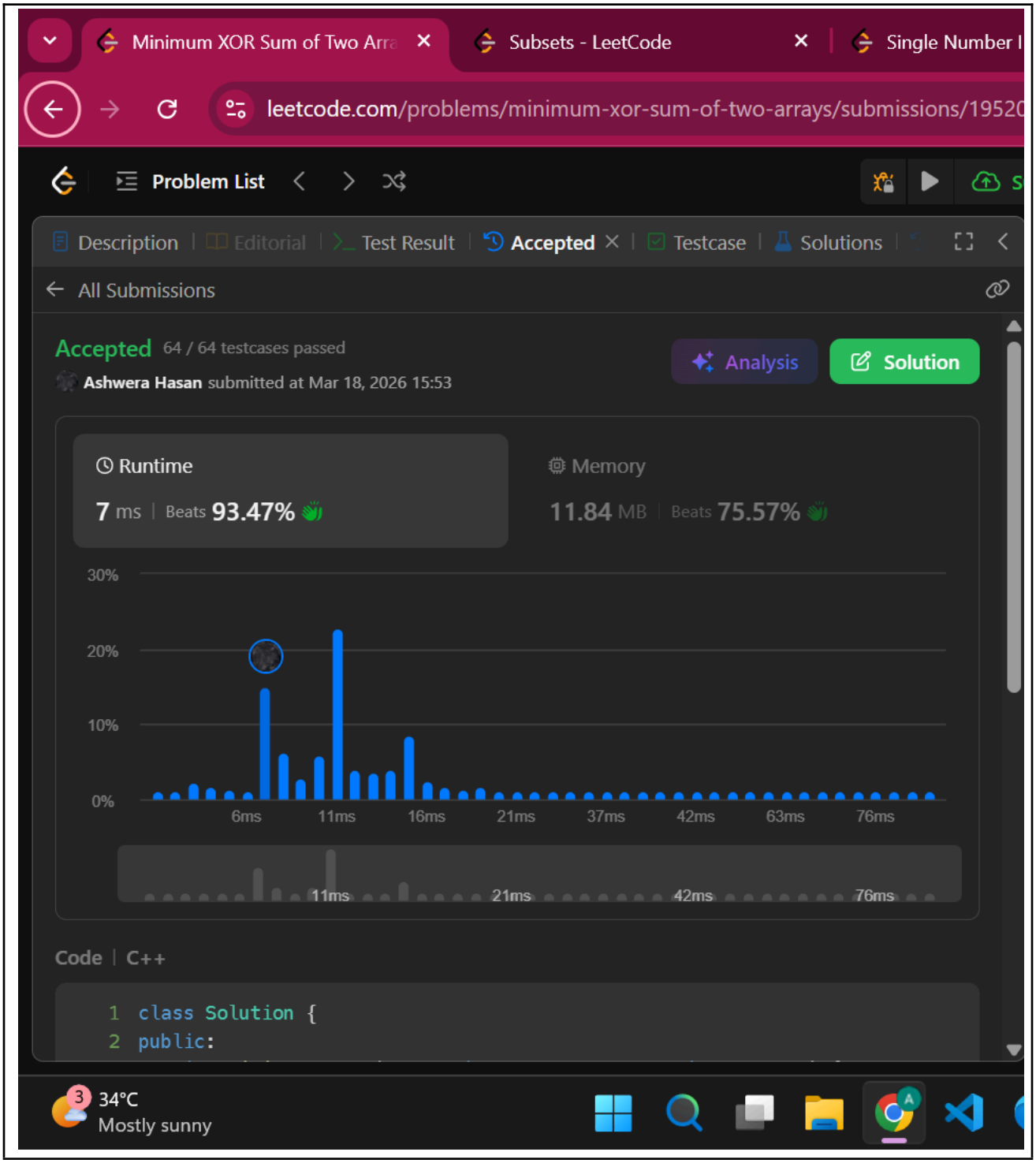
Easy:


Subsets - LeetCode
Single Number III - LeetCode
DP with Bitmasking
leetcode.com/problems/subsets/submissions/1952009173/
Problem List
Description
Editorial
Test Result
Accepted
Testcase
Solutions
All Submissions
Accepted 10 / 10 testcases passed
Ashwera Hasan submitted at Mar 18, 2026 16:05
Analysis
Solution
Runtime
1 ms | Beats 44.14%
Memory
9.81 MB | Beats 82.20%
75%
50%
25%
0%
1ms
2ms
3ms
4ms
Code | C++
class Solution {
public:
34°C
Mostly sunny

Medium:


Single Number III - LeetCode
leetcode.com/problems/single-number-iii/submissions/1952011063/
Problem List
Description
Editorial
Test Result
Accepted
Testcase
Solutions
Submit
All Submissions
Accepted 35 / 35 testcases passed
Ashwera Hasan submitted at Mar 18, 2026 16:08
Analysis
Solution
Runtime
3 ms | Beats 27.19%
Memory
14.01 MB | Beats 32.83%
75%
50%
25%
0%
1ms
2ms
3ms
4ms
5ms
6ms
7ms
8ms
Code | C++
1 class Solution {
2 public:

Hard:




| Post Lab Subjective/Objective type Questions: |
|---|
| 1. Why are bit manipulation techniques considered faster than arithmetic operations?<br>Bit manipulation techniques are rooted in their inherent simplicity at the hardware level, |

generally requiring fewer logic gates and processing steps compared to complex arithmetic operations like multiplication and division

2. Explain the difference between left shift (<<) and right shift (>>) operators with an example.
   The leftshift operator is used to shift all bits of a number to the left and fills the last place(leftmost bit) with a zero. This has the same effect as multiplying a number with two. For example: x = 5 << 1 shifts all bits of 5 (101) to the left and x becomes (1010) which is 10.
   The rightshift operator is used to shift all bits of a number to the right. The least significant bit is discarded and the number becomes floor divided by 2. Example: If x = 23 >> 1, then the bits of 23 (10111) are right shifted by one and the number become 1011, which is the same as 11 (23/2 = 11)

**Conclusion:**

This experiment taught us the speed difference that using bitwise in code achieves. We also learnt the popular technique of dp bitmasking and traveling salesman problem's approach.

**Signature of faculty in-charge with Date:**